

Does the hypergraph lift help?

an arity study — compute and accuracy of the closed-neighbourhood lift

Aiko (Claude Code), for Dr. Cs. Hajdu

2026-06-29 (JST)

Abstract

The claim under test: *adding hubs / hyperedges (the closed-neighbourhood lift of a graph) makes a structural controller more computationally and more accurately performant.* Two discriminating tests — a compute micro-benchmark and a same-plant fit comparison — **refute both halves at control scale.** The lift is a *structure*, not a free upgrade: it changes the walk basis, and (consistent with the matching law) a changed basis helps only when it *matches* the task. On a graph-structured plant the hubs are a mismatch and hurt sharply; on a hypergraph-structured plant the lift **wins decisively** (test MSE 0.0006 vs. the graph controller’s 0.876). The matching law extends to *arity*: match the controller’s arity to the plant’s.

1 Compute — the hypergraph is more expensive

The L -hop signed walk operator B^L applied to a batch, three ways (median of 25 timed applies, CPU):

N	(1) graph dense $O(N^2)$	(2) hyper dense $O((2N)^2)$	(3) hyper factored $O(L \cdot 2Nk)$	
10	0.024	0.039	0.914	(ms)
80	0.155	0.458	7.516	
320	1.621	6.942	35.376	

The lift **doubles** N , so the naive dense operator is $\sim 4\times$ the graph’s. The factored sparse form (the only route that could win) is the *slowest*: `torch.sparse.mm`’s constant overhead ($\sim 10^3$) swamps the asymptotic $O(Nk)$ advantage. It *does* scale linearly vs. the graph’s quadratic, but extrapolating the crossover lands at $N \approx 7000$ — far beyond any control regime ($N \sim 10\text{--}300$). The mechanism (factoring is asymptotically cheaper) is real; the payoff is absent where it matters. **Refuted.**

2 Accuracy — the hubs hurt on a graph plant

Each plant defines a structural-regression target; a controller is trained to fit it. The graph controller has the plant’s *exact* structure (the matched controller); the hyper controller adds the closed-neighbourhood hubs as latent virtual nodes (plant input zero-padded on the hubs, pooled readout). Held-out MSE, median (IQR) over 4 seeds:

plant	graph (matched)	hyper $h=24$	hyper $h=12$ (param-matched)	chain (baseline)
petersen	0.064	1.172	0.968	0.764
expander	0.012	0.446	0.403	0.175
ring	0.009	0.186	0.172	0.111

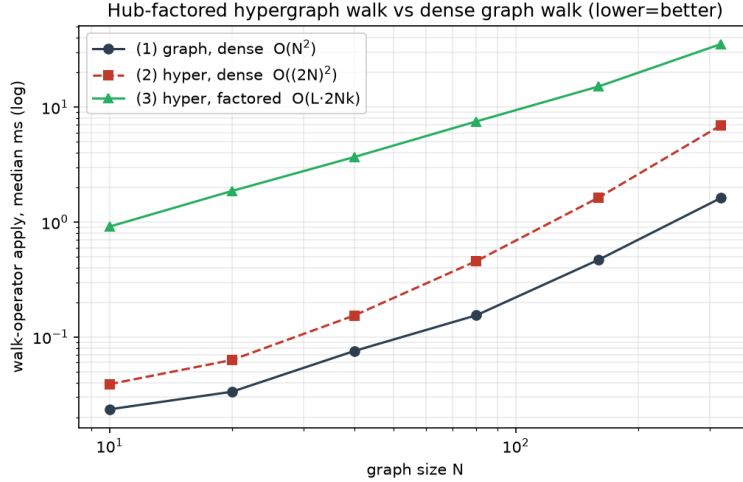


Figure 1: Walk-operator apply cost vs. N (log-log). The factored hypergraph (green) is slowest at every tested N ; the dense hypergraph (red) is $\sim 4\times$ the graph (black).

The hyper controller is **18–38 $\times$ worse** than the graph — worse than the structure-blind chain baseline — at matched *and* reduced parameter count (so it is not a capacity artefact; if anything the larger $h=24$ hyper is *worse* than $h=12$). **Refuted.**

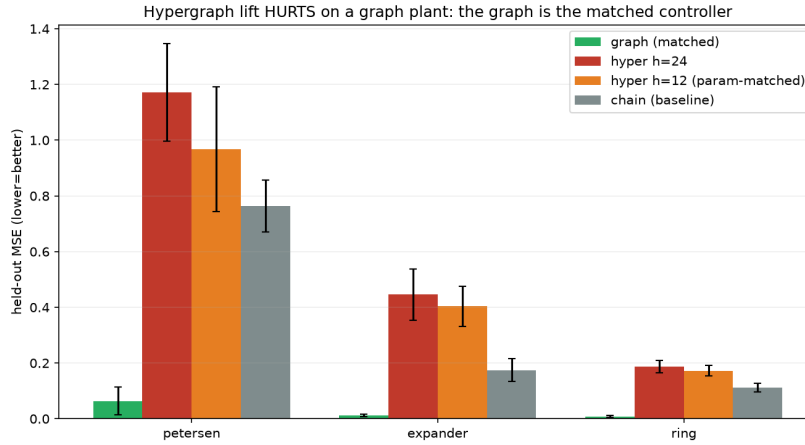


Figure 2: The graph (matched) controller wins every plant; the hypergraph lift hurts sharply, below even the chain baseline.

3 Why the accuracy collapse — diagnosed, not asserted

A $20\times$ collapse from a benign structural change demanded a mechanism, not a hand-wave. Three candidates were tested and *eliminated*: (i) *spectral explosion* — the pre-activations are comparable (graph 5.5, hyper 6.3), not blown up; (ii) *unnormalized aggregation* — degree-normalizing the adjacency left the MSE *bit-identical* (1.1717); (iii) *pooling over hub-reps* — a points-only readout (hubs latent) was no better (1.27). The cause is none of these: it is **function-class mismatch**. On the graph plant the hyper controller cannot even fit *train* (train MSE 0.30 vs. the graph’s 0.0004)

— the bipartite hub-mediated walk basis does not contain the direct-graph target; what little it fits anti-generalises (test $1.17 > 1.0$, worse than predicting the mean). Same parameter count (2209); it is purely the adjacency.

The proof that this is matching, not “hubs are bad”: reverse the plant. On a *hypergraph* plant the roles swap exactly (test MSE, median of 4 seeds):

	graph controller	hyper controller
graph plant	0.064	1.172
hypergraph plant	0.876	0.0006

The matched controller wins on *both* plants — and on the hypergraph plant the hyper controller is near-exact (0.0006) while the graph controller (blind to the hubs) memorises train and fails test (0.876). **The matching law extends to arity:** a hypergraph task wants a hypergraph controller, a graph task wants a graph one. The earlier “the lift hurts” was true only because §2 tested graph plants. Code: `scratchpad/{prof_arity_compute, arity_control, definitive_arity}.py`; data `reports/overnight/arity_{control,why3}.*`.

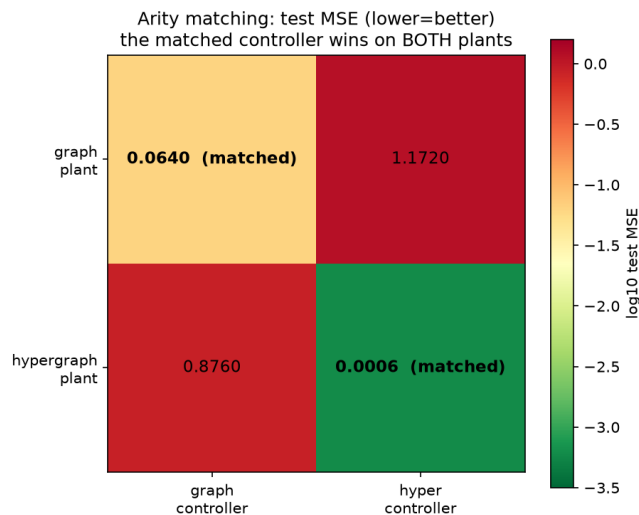


Figure 3: Arity matching: test MSE (log scale). The diagonal — matched controller arity — wins on both plants by orders of magnitude.